

7

The Four-Step Process of Causal Inference

Welcome to *Chapter 7*!

This is a true milestone in our journey. In this chapter, we'll learn how to neatly structure the entire causal inference process using the DoWhy library (Sharma & Kiciman, 2020). By the end of this chapter, you'll be able to write production-ready causal inference pipelines using linear and non-linear estimators.

We'll start with an introduction to DoWhy and its sister library, EconML (Battochi et al., 2019). After that, we'll see how to use the **graph modeling language (GML)**, which we introduced briefly in *Chapter 4* to translate our assumptions regarding the data-generating process into graphs. Then, we'll see how to compute causal **estimands** and causal **estimates** using DoWhy. Finally, we'll introduce **refutation tests** and see how to apply them to our models. We'll conclude the chapter with an example of a complete causal inference process. By the end of this chapter, you'll have a solid understanding of the mechanics of the causal inference process and be able to carry it out for your own problems.

In this chapter, we'll cover the following topics:

- Introduction to DoWhy and EconML
- Practical GML and graphs
- Causal estimands and estimates in DoWhy
- Refutation tests
- A full causal process example using DoWhy and EconML

Introduction to DoWhy and EconML

In this section, we'll introduce the DoWhy and EconML packages.

We'll start with an overview of the Python causal ecosystem and then discuss what DoWhy and EconML are.

Then, we'll share why they are the packages of choice for this book.

Finally, we'll dive deeper into DoWhy's APIs and look into the integration between DoWhy and EconML.

Yalla!

Python causal ecosystem

The Python causal ecosystem is dynamically expanding. It is becoming increasingly rich and powerful. At the same time, it can also be confusing, especially when you're just starting your causal journey.

The following list presents a selection of actively developed Python causal packages that I am aware of at the time of writing this book:

- **CATENets**: A package implementing a number of neural network-based conditional average treatment effect estimators in JAX and PyTorch. We introduce CATENets in *Chapter 11*: <https://github.com/AliciaCurth/CATENets>.
- **causal-learn**: A causal discovery package from Carnegie-Mellon University. It is a Python translation and extension of the famous Java library Tetrad. It includes implementations of a number of conditional independence tests: <https://github.com/cmu-phil/causal-learn>.
- **causalimpact**: This package is a port to its R counterpart. It uses structural Bayesian models to estimate causal effects in quasi-experimental time series data: <https://github.com/jamalsenouci/causalimpact>.
- **causal inference**: A causal inference package implementing a number of basic causal estimators: <https://causalinferenceinpython.org/>.
- **causal lib**: This package provides a suite of causal methods, encapsulated in an sklearn-style API. It includes meta-algorithms and an evaluation suite. It is an intuitive and flexible API that is supported by IBM: <https://github.com/IBM/causal lib>.
- **CausalPy**: Four popular quasi-experimental methods implemented on top of the PyMC Python Bayesian framework. The package offers synthetic control, interrupted time series, difference-in-differences, and regression discontinuity methods: <https://github.com/pymc-labs/CausalPy>.

- CausalML: A library for uplift modeling and causal inference backed by Uber. The authors declare that it's stable and incubated for long-term support. Algorithms can be integrated into DoWhy's flow: <https://github.com/uber/causalml>.
- Causica: A package implementing an end-to-end causal algorithm, DECI. We discuss DECI in *Part 3, Causal Discovery*: <https://github.com/microsoft/causica>.
- CDT: A library with a broad selection of causal discovery methods. Some methods are ported from R packages. At the time of writing (September 2022), it does not include methods created later than 2019: <https://fentechsolutions.github.io/CausalDiscoveryToolbox/html/index.html>.
- Differences: A package that implements a number of quasi-experimental techniques based on the difference-in-differences technique: <https://github.com/bernardodionisi/differences>.
- DoubleML: A Python and R package implementing a set of methods based on **double machine learning (DML)**: <https://docs.doubleml.org/stable/index.html>.
- DoWhy: A complete framework for DAG-based causal inference. Causal discovery is on the roadmap: <https://py-why.github.io/dowhy>.
- EconML: A library focused on modeling heterogeneous treatment effects using machine learning. It is similar in scope to CausalML. It is supported by Microsoft and deeply integrated with DoWhy: <https://github.com/microsoft/EconML>.
- gCastle: A comprehensive library for causal discovery. It contains implementations of many classic causal discovery algorithms as well as some of the most recent ones. It was developed by Huawei's Noah's Ark Lab. We'll use it in *Part 3, Causal Discovery*: <https://github.com/huawei-noah/trustworthyAI/tree/master/gcastle>.
- GRAPL (grap1-causal): A computational library for working with causal graphs. It includes advanced identification algorithms, such as **directed acyclic graphs (DAGs)**: <https://github.com/max-little/GRAPL>.
- LiNGAM: A causal discovery package implementing a number of LiNGAM-family algorithms: <https://lingam.readthedocs.io/en/latest/index.html>.
- PySensemakr: A Python implementation of the R sensemakr library. It implements a number of causal sensitivity analysis tools for regression models: <https://github.com/nlapier2/PySensemakr>.
- Semopy: A package for structural equation modeling in Python: <https://semopy.com/>.
- scikit-uplift: A library focused on uplift modeling using an sklearn-style API: https://www.uplift-modeling.com/en/latest/user_guide/index.html.

- `tfcausalimpact`: A native Python implementation of R's `causalimpact` built on top of TensorFlow Probability. It provides a flexible framework for estimating causal effects in quasi-experimental time series data using structural Bayesian models: <https://github.com/WillianFuks/tfcausalimpact>.
- `YLearn`: A package containing a mixture of causal methods, such as a number causal effect estimators, one causal discovery algorithm (at the time of writing), and other utilities: <https://ylearn.readthedocs.io/en/latest/>.

As you can see, we have many options to choose from. You might be wondering – what's so special about DoWhy and EconML that we've chosen to use these particular packages in the book?

Why DoWhy?

There are at least six reasons why I think choosing DoWhy as the foundation of your causal ecosystem is a great idea. Let me share them with you:

- DoWhy offers a well-designed, consistent, and practical API
- DoWhy is designed in a way that enables you to run the entire causal inference process in *four clearly defined and easily reproducible steps*
- DoWhy integrates smoothly with many other libraries (such as EconML, scikit-learn, and CausalML)
- DoWhy is actively maintained by a team of seasoned researchers and developers
- DoWhy is supported by organizations such as Microsoft and AWS, which increases the chances for long-term stable development
- DoWhy looks into the future with a frequently updated roadmap

Oui, mon ami, but what is DoWhy?

The official documentation (<https://bit.ly/DoWhyDocs>) describes DoWhy as an *end-to-end causal inference library*. I like to think of it as a framework. DoWhy offers a comprehensive toolkit for causal inference. In most cases, you won't need to go for another package to address your causal inference problem.

With the new experimental GCM API (Blobaum et al., 2022; for more details, see <https://bit.ly/DoWhyGCM>), DoWhy becomes even more powerful! Another great thing about DoWhy is that it translates a complex process of causal inference into a set of simple and easily reproducible steps.

DoWhy's API zoo

The version of DoWhy that we use across the book (0.8) offers three ways to work with causal models: the main API based on the `CausalModel` class; the high-level pandas API, which allows you to perform interventions directly on pandas DataFrames; and an experimental GCM API that allows us to easily estimate causal effects and compute interventions, counterfactuals, and attribute distributional changes between datasets in just a couple lines of code.

How about EconML?

EconML is a Python package built as a part of Microsoft Research's *ALICE* project (<https://bit.ly/MsftAlice>). Its main goal is to estimate causal effects from observational data via machine learning (Battochi et al., 2019). The package provides us with a scikit-learn-like API for fitting causal machine learning estimators. On top of this, we get a series of very useful methods for obtaining causal effects, confidence intervals, and integrated interpretability tools such as tree interpreters or Shapley values.

One of the great things about EconML is that it's deeply integrated with DoWhy. This integration allows us to call EconML estimators from within DoWhy code without even explicitly importing EconML. This is a great advantage, especially when you're in the early experimental stage of your project and your code does not have a strong structure. The integration allows you to keep your code clean and compact and helps to avoid excessive clutter.

Great, now that we've had a short introduction, let's jump in and see how to work with DoWhy and EconML. We'll learn more about both libraries on the way.

In the next four sections, we'll see how to use the main DoWhy API to perform the following four steps of causal inference:

1. Modeling the problem
2. Finding the estimand(s)
3. Computing the estimates
4. Validating the model

Before we start, let's generate some data.

We're going to use the `GPSMemorySCM` class from *Chapter 6* for this purpose. The code for this chapter is in the `Chapter_07.ipynb` notebook (<https://bit.ly/causal-ntbk-07>).

Let's initialize our SCM, generate 1,000 observations, and store them in a data frame:

```
scm = GPSMemorySCM()
gps_obs, hippocampus_obs, memory_obs = scm.sample(1000)

df = pd.DataFrame(np.vstack([gps_obs, hippocampus_obs,
                             memory_obs]).T, columns=['X', 'Z', 'Y'])
```

Note that we denoted the columns for GPS with X , hippocampal volume with Z , and spatial memory with Y .

Naturally, in most real-world scenarios, we'll work with real-world data rather than generated data. Nonetheless, in both cases, it's important to keep your naming conventions clean and consistent between your data frame, your graph, and your `CausalModel` object. Any inconsistencies might result in distorted or unexpected results.

Great, we're now ready to take the first step!

Step 1 – modeling the problem

In this section, we'll discuss and practice step 1 of the four-step causal inference process: *modeling the problem*.

We'll split this step into two substeps:

1. Creating a graph representing our problem
2. Instantiating DoWhy's `CausalModel` object using this graph

Creating the graph

In *Chapter 3*, we introduced a graph language called GML. We'll use GML to define our data-generating process in this section.

Figure 7.1 presents the GPS example from the previous chapter, which we'll model next. Note that we have omitted variable-specific noise for clarity:

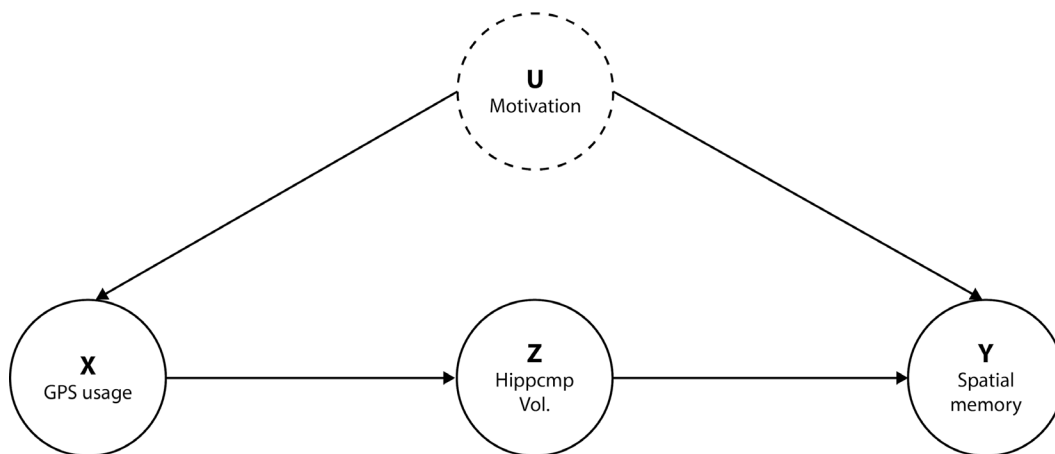


Figure 7.1 – The graphical model from Chapter 6

Note that the graph in *Figure 7.1* contains an unobserved variable, U . We did not include this variable in our dataset (it's unobserved!), but we'll include it in our graph. This will allow DoWhy to recognize that there's an unobserved confounder in the graph and find a relevant estimand for us automatically.

Great! Let's translate the model from *Figure 7.1* into a GML graph:

```
gml_graph = ""
graph [
  directed 1

  node [
    id "X"
    label "X"
  ]
  node [
    id „Z"
    label „Z"
  ]
  node [
    id "Y"
    label "Y"
  ]
  node [
    id „U"
    label „U"
  ]

  edge [
    source "X"
    target "Z"
  ]
  edge [
    source "Z"
    target "Y"
  ]
  edge [
    source "U"
    target "X"
  ]
  edge [
    source "U"
    target "Y"
  ]
]
```

Our definition starts with the `directed` keyword. It tells the parser that all edges in the graph should be directed. To obtain an undirected graph, you can use `undirected` instead.

Next, we define the nodes. Each node has a unique ID and a label. Finally, we define the edges. Each edge has a source and a target. The entire definition is encapsulated in a Python multiline string.

Let's put our graph into action!

Building a CausalModel object

To instantiate a `CausalModel` object for our problem, we need to provide the constructor with four things: data as a pandas data frame, a GML graph, the name of the treatment variable in the data frame, and the name of the outcome variable in the data frame.

We denoted GPS – our treatment – with *X*. Our outcome – memory change – was denoted by *Y*. Our data is represented by a pandas data frame, *df*, and our graph is assigned to the *gml_graph* variable.

We're ready to instantiate `CausalModel`:

```
model = CausalModel(  
    data=df,  
    treatment='X',  
    outcome='Y',  
    graph=gml_graph  
)
```

To make sure everything works as expected, let's plot the model:

```
model.view_model()
```

This should result in a visualization similar to the one in *Figure 7.2*:

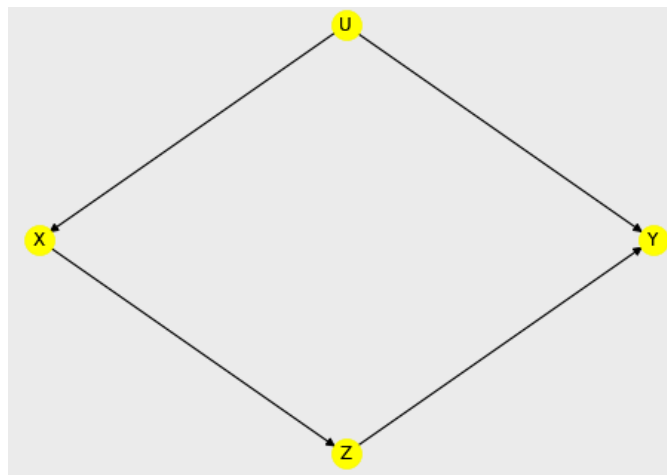


Figure 7.2 – A visualization of our model

In this section, we’ve learned how to create a GML graph to model our problem and how to pass this graph into the `CausalModel` object. In the next section, we’ll see how to use DoWhy to automatically find estimands for our graph.

Step 2 – identifying the estimand(s)

This short section is all about finding estimands with DoWhy. We’ll start with a brief overview of estimands supported by the library and then jump straight into practice!

DoWhy offers three ways to find estimands:

- Back-door
- Front-door
- Instrumental variable

We know all of them from the previous chapter. To see a quick practical introduction to all three methods, check out my blog post *Causal Python — 3 Simple Techniques to Jump-Start Your Causal Inference Journey Today* (Molak, 2022; <https://bit.ly/DoWhySimpleBlog>).

Let’s see how to use DoWhy in order to find a correct estimand for our model.

It turns out it is very easy! Just see for yourself:

```
estimand = model.identify_effect()
```

Yes, that’s all!

We just call the `.identify_effect()` method of our `CausalModel` object and we’re done!

Let’s print out our estimand to see what we can learn:

```
print(estimand)
```

This results in the following output:

```
Estimand type: nonparametric-ate

### Estimand : 1
Estimand name: backdoor
No such variable(s) found!

### Estimand : 2
Estimand name: iv
No such variable(s) found!

### Estimand : 3
```

```

Estimand name: frontdoor
Estimand expression:
Expectation(Derivative(Y, [Z])*Derivative([Z], [X]))
Estimand assumption 1, Full-mediation: Z intercepts (blocks) all
directed paths from X to Y.
Estimand assumption 2, First-stage-unconfoundedness: If  $U \rightarrow \{X\}$  and
 $U \rightarrow \{Z\}$  then  $P(Z|X, U) = P(Z|X)$ 
Estimand assumption 3, Second-stage-unconfoundedness: If  $U \rightarrow \{Z\}$  and  $U \rightarrow Y$ 
then  $P(Y|Z, X, U) = P(Y|Z, X)$ 

```

We see that DoWhy prints out three different estimands. There's the estimand's name and information if a given type of estimand has been found for our model.

For the estimand that has been found, there's information about it printed out; for estimands that haven't been found, there's a `No such variable(s) found!` string printed out.

You can see that DoWhy has found only one estimand for our graph: `frontdoor`. As you might remember from the previous chapter, this is the correct one for our model!

Before we finish this section, I want to reiterate one thing. The capabilities of DoWhy are really great, yet it will only be able to find estimands for us if our model is identifiable using one of the three supported methods. For more advanced identification strategies, check out the **grapl-causal** library created by Max Little of the University of Birmingham and MIT (<https://bit.ly/GRAPLCausalRepo>).

In this section, we learned how to find estimands automatically using DoWhy. Now, we're ready to compute the estimates!

Step 3 – obtaining estimates

In this section, we'll compute causal effect estimates for our model.

Computing estimates using DoWhy is as simple as it can be. To do it, we need to call the `.estimate_effect()` method of our `CausalModel` object:

```

estimate = model.estimate_effect(
    identified_estimand=estimand,
    method_name='frontdoor.two_stage_regression')

```

We pass two arguments to the method:

- Our identified estimand
- The name of the method that will be used to compute the estimate

You might recall from *Chapter 6* that we needed to fit two linear regression models, get their coefficients, and multiply them in order to obtain the final causal effect estimate. DoWhy makes this process much easier for us.

Let’s print out the result:

```
print(f'Estimate of causal effect (
      linear regression): {estimate.value}')
```

This gives us the following output:

```
Estimate of causal effect (linear regression): -0.4201729192182768
```

That’s almost identical to the true causal effect of -0.42!

Note that we haven’t set the random seed, so your results might slightly differ.

Currently, DoWhy only supports one estimator for the front-door criterion (`frontdoor.two_stage_regression`), but many more estimators are available in general. We’ll introduce some of them soon.

In this short section, we learned how to compute causal estimates using DoWhy.

Let’s think about how to validate our causal model.

Step 4 – where’s my validation set? Refutation tests

In this section, we’ll discuss ideas regarding causal model validation. We’ll introduce the idea behind refutation tests. Finally, we’ll implement a couple of refutation tests in practice.

How to validate causal models

One of the most popular ways to validate machine learning models is through **cross-validation (CV)**. The basic idea behind CV is relatively simple:

1. We split the data into k folds (subsets).
2. We train the model on $k - 1$ folds and validate it on the remaining fold.
3. We repeat this process k times.
4. At every step, we train on a different set of $k - 1$ folds and evaluate on the remaining fold (which is also different at each step).

Figure 7.3 presents a schematic visualization of a five-fold CV scheme:

Iteration 1				
Iteration 2				
Iteration 3				
Iteration 4				
Iteration 5				

Figure 7.3 – Schematic of five-fold CV

In Figure 7.3, the blue folds denote validation sets, while the white ones denote training sets. We train a new model at each iteration using white folds for training and evaluate using the remaining blue fold. We collect metrics over iterations and compute a statistic summary (most frequently, the average).

CV is a widely adopted method to estimate prediction errors in statistical learning (yet it turns out that it's much less understood than we tend to think; Bates et al., 2021).

It would be great if we could use CV for causal models as well! Its simplicity and accessibility of implementations would make model evaluation really easy!

Unfortunately, this approach cannot provide us with any guarantees regarding the causal structure of the model (although it can be useful when working with and evaluating causal estimators, assuming that we know the correct causal structure; more on this in *Chapter 10*).

Why?

Although CV can provide us with some information about the estimator fit, this approach will not work in general as a causal model evaluation technique. The reason is that CV operates on rung 1 concepts. Let's think of an example.

Do you recall **Markov equivalence classes (MECs)**, which we discussed in *Chapter 5*? All graphs within one MEC encode the same statistical independence structure. One consequence of this is that unless we intervene on critical nodes, all graphs belonging to the same MEC could generate virtually the same observational data. This means that we could have a wrong causal model that behaves very well in terms of CV prediction errors. This is problematic. Is there anything we can do to mitigate the limitation of CV?

Introduction to refutation tests

In 1935, the Austrian philosopher Karl Popper published his seminal book *Logik der Forschung* (Popper, 1935; rewritten English version: Popper, 1959). In the book, Popper wrestles with the idea of induction – generalizing knowledge from a finite sample of observations. He concludes (following our friend David Hume from *Chapter 1*) that sentences such as *all Xs are A* that are trying to capture the underlying principles of some phenomenon of interest can never be unambiguously proven. For instance, the sentence *the sun always rises in the east* might always be true for me in my lifetime, yet we can imagine a scenario where one day it does not. We have never observed this scenario, but it is physically possible.

Compare this example with another one: *the train always passes the station within five minutes after I hear the morning weather forecast on the radio* (do you remember Hume’s association-based definition of causality?). Popper says that we never observe all Xs and therefore it’s impossible to *prove* a theory. He finds this problematic because he wants to see a difference between science and non-science (this is known as the **demarcation problem**). To solve this tension, Popper proposes that instead of proving a theory, we can try to disprove it.

His logic is simple. He says that “all Xs are A” is logically equivalent to “no X is not-A.” Therefore, if we find X that is not A, we can show that the theory is false. Popper proposes that scientific theories are *falsifiable* – we can define a set of conditions in which the theory would fail. On the other hand, non-scientific theories are *non-falsifiable* – for instance, the existence of a higher intelligent power that cannot be quantified in any systematic way.

Can Popperian ideas help us with evaluating causal models?

Let’s get Popperian!

Every statistical model can be thought of as a hypothesis regarding some process. In particular, a *causal model* is a hypothesis regarding some *data-generating process*. In this sense, any model is an embodiment of a micro-theory of some real-world phenomenon and such a theory can be falsified.

Refutation tests aim to achieve this by modifying the model or the data. There are two types of transformations available in DoWhy (Sharma & Kiciman, 2020):

- Invariant transformations
- Nullifying transformations

Invariant transformations change the data in such a way that the result should not change the estimate. If the estimate changes significantly, the model fails to pass the test.

Nullifying transformations change the data in a way that should cause the estimated effect to be zero. If the result significantly differs from zero, the model fails the test.

The basic idea behind refutation tests is to modify an element of either the model or a dataset and see how it impacts the results.

For instance, a *random common cause refuter* adds a new confounding variable to the dataset and controls for it. If the original model is correctly specified, we expect that such an addition will *not lead to significant changes* in the model estimates (therefore, this test belongs to the *invariant transformations* category). We'll see a couple of refutation tests in action in this chapter.

Now, let's see how one of them – the *data subset refuter* – works.

Let's refute!

Let's apply some refutation tests to our model. Note that in DoWhy 0.8, not all tests will work with front-door estimands.

We apply `data_subset_refuter` in the following way:

```
refute_subset = model.refute_estimate(  
    estimand=estimand,  
    estimate=estimate,  
    method_name="data_subset_refuter",  
    subset_fraction=0.4)
```

This test removes a random subset of the data and re-estimates the causal effect. In expectation, the new estimate (on the subset) should not significantly differ from the original one. Let's print out the results:

```
print(refute_subset)
```

This gives us the following output:

```
Refute: Use a subset of data  
Estimated effect:-0.4201729192182768  
New effect:-0.41971603098814647  
p value:0.98
```

As you can see, the original and newly estimated effects are very close and the p -value is high, indicating that there's likely no true difference between the two estimates. This result does not falsify our hypothesis and perhaps makes us a bit more confident that our model might be correct.

In this section, we discussed the basic challenges of validating causal models. We introduced the logic behind refutation tests and learned how to apply them in practice using DoWhy.

In the next section, we'll practice our DoWhy causal inference skills and see more refutation tests in action.

Full example

This section is here to help us solidify our newly acquired knowledge. We'll run a full causal inference process once again, step by step. We'll introduce some new exciting elements on the way and – finally – we'll translate the whole process to the new GCM API. By the end of this section, you will have the confidence and skills to apply the four-step causal inference process to your own problems.

Figure 7.4 presents a graphical model that we'll use in this section:

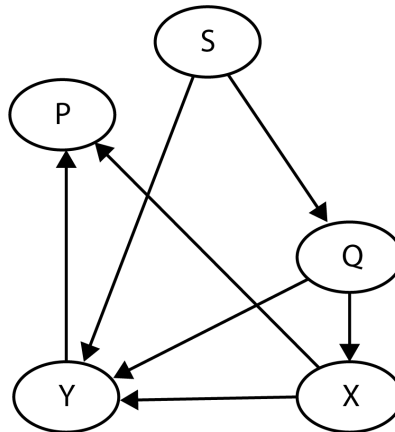


Figure 7.4 – A graphical model that we'll use in this section

We'll generate 1,000 observations from an SCM following the structure from *Figure 7.4* and store them in a data frame:

```

SAMPLE_SIZE = 1000

S = np.random.random(SAMPLE_SIZE)
Q = 0.2*S + 0.67*np.random.random(SAMPLE_SIZE)
X = 0.14*Q + 0.4*np.random.random(SAMPLE_SIZE)
Y = 0.7*X + 0.11*Q + 0.32*S +
    0.24*np.random.random(SAMPLE_SIZE)
P = 0.43*X + 0.21*Y + 0.22*np.random.random(SAMPLE_SIZE)

# Build a data frame
df = pd.DataFrame(np.vstack([S, Q, X, Y, P]).T,
                  columns=['S', 'Q', 'X', 'Y', 'P'])

```

Great, now we're ready to build a graph.

Step 1 – encode the assumptions

As the graph in *Figure 7.4* is big enough to make writing its GML definition manually daunting, we'll automate it with two simple `for` loops. Feel free to steal this trick and use it in your own projects:

```
nodes = ['S', 'Q', 'X', 'Y', 'P']
edges = ['SQ', 'SY', 'QX', 'QY', 'XP', 'YP', 'XY']

gml_string = 'graph [directed 1\n'

for node in nodes:
    gml_string += f'\tnode [id "{node}" label "{node}"]\n'

for edge in edges:
    gml_string += f'\tedge [source "{edge[0]}" target\n'
                        "{edge[1]}" ]\n'

gml_string += '']
```

Let's unpack this code quickly:

1. First, we define two lists: a list of nodes (`nodes`) and a list of edges (`edges`).
2. Next, we create the `gml_string` variable, which contains the opening bracket, `[`, the `directed` keyword, and our graph's ID (number 1).
3. Then, we run the first `for` loop over the nodes. At each step, we append a new line to our `gml_string` representing a node and specifying its ID and label (we've also added tabulation and newlines to make it more readable, but this is not necessary).
4. After the first loop is done, we run the second `for` loop over edges. We append a new line at each step again. This time, each line contains information about the edge source and target.
5. Finally, we append the closing bracket, `]`, to our `gml_string`.

Let's print out our definition to see whether it's as expected:

```
print(gml_string)
```

This gives us the following output:

```
graph [directed 1
  node [id "S" label "S"]
  node [id "Q" label "Q"]
  node [id "X" label "X"]
  node [id "Y" label "Y"]
  node [id "P" label "P"]
  edge [source "S" target "Q"]
```



```
edge [source "S" target "Y"]  
edge [source "Q" target "X"]  
edge [source "Q" target "Y"]  
edge [source "X" target "P"]  
edge [source "Y" target "P"]  
edge [source "X" target "Y"]  
]
```

Beautiful! The GML part is correct.

Let's instantiate the `CausalModel` object:

```
model = CausalModel(  
    data=df,  
    treatment='X',  
    outcome='Y',  
    graph=gml_string  
)
```

Let's visualize the model to make sure that our graph's structure is as expected:

```
model.view_model()
```

You should see a plot similar to the one in *Figure 7.5*:

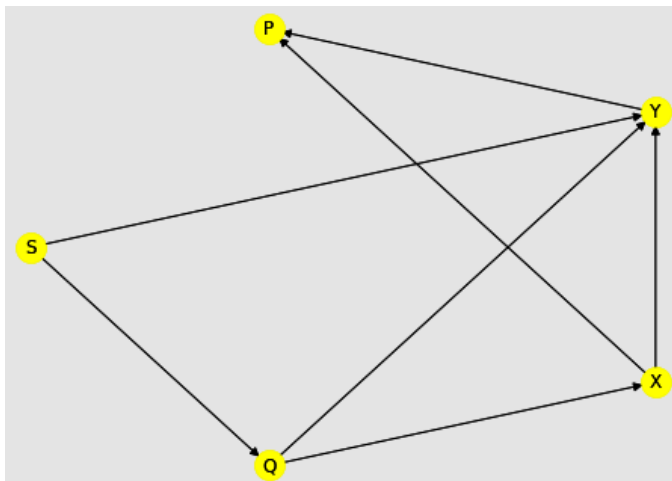


Figure 7.5 – A visualization of our DoWhy CausalModel

The graph in *Figure 7.5* has the same set of nodes and edges that the graph in *Figure 7.4* has. It confirms that our GML definition is correct.

We're now ready to find the estimand(s) for our model.

Step 2 – getting the estimand

Before we move on to coding, can you figure out what estimand we expect here? How many confounders are there between X and Y ? What will happen if we control for Q ? Or for P ? Or for S ?

Let's see what DoWhy's view on this is:

```
estimand = model.identify_effect()
print(estimand)
```

The estimand output is the following:

```
Estimand type: nonparametric-ate

### Estimand : 1
Estimand name: backdoor
Estimand expression:
  d
  ——— (E[Y|Q])
  d[X]
Estimand assumption 1, Unconfoundedness: If  $U \rightarrow \{X\}$  and  $U \rightarrow Y$  then
 $P(Y|X, Q, U) = P(Y|X, Q)$ 

### Estimand : 2
Estimand name: iv
No such variable(s) found!

### Estimand : 3
Estimand name: frontdoor
No such variable(s) found!
```

We can see that DoWhy proposed one valid estimand – backdoor. Although our graph looks a bit complex, it contains only one back-door path. Controlling for Q deconfounds the relationship between X and Y .

Step 3 – estimate!

Although our SCM is pretty simple and we could model it using linear regression, we'll use a more advanced estimator this time. Along the way, we'll learn how to leverage DoWhy's integration with other packages in the Python machine learning ecosystem to build advanced multicomponent estimators.

First, let's import some models from scikit-learn. We picked LassoCV and GradientBoostingRegressor. Both are rather overkill for our simple problem, but we picked them on purpose so we can see DoWhy's flexibility and integration capabilities in action:

```
from sklearn.linear_model import LassoCV
from sklearn.ensemble import GradientBoostingRegressor
```

Now, let's build an estimator.

We will use a DML estimator from the EconML package. DML is a family of methods for estimating causal effects that was originally proposed by Victor Chernozhukov and colleagues (Chernozhukov et al., 2016).

Behind the scenes, DML fits *three machine learning models* to compute de-biased estimates of treatment effects. First, it predicts the outcome from the controls. Then, it predicts the treatment from the controls. Finally, it fits the model that regresses residuals from the second model on the residuals from the first model.

We will discuss DML in greater detail in *Chapter 10*.

Let's see how to leverage the power of DML using DoWhy:

```
estimate = model.estimate_effect(
    identified_estimand=estimand,
    method_name='backdoor.econml.dml.DML',
    method_params={
        'init_params': {
            'model_y': GradientBoostingRegressor(),
            'model_t': GradientBoostingRegressor(),
            'model_final': LassoCV(fit_intercept=False),
        },
        'fit_params': {}
    }
)
```

Let's unpack this code:

1. First, we provided the `estimate_effect()` method with the *estimand*.
2. After that, we pointed to what method we want to use. Here, we've chosen one of EconML's DML estimators. As you can see, we haven't even imported EconML in our session. We passed a `'backdoor.econml.dml.DML'` string instead. This is an example of the deep integration between DoWhy and EconML.
3. Then, we defined the models we want to use for each of the three stages of DML estimation. We use regression models only as our treatment and outcome are both continuous.
4. We left the `fit_params` dictionary empty because we didn't want to specify any initial parameters for our models.

Let's see how well DML handled the task:

```
print(f'Estimate of causal effect (DML): {estimate.value}')
```

This results in the following output:

```
Estimate of causal effect (DML): 0.6998599202773215
```

Good job! The true effect is 0.7 and so we are really close!

Before we move further, let's see a simpler model in action.

A simple linear regression would likely be sufficient to model our problem in this section.

For the sake of comparison, let's see the results of linear regression:

```
estimate_lr = model.estimate_effect(  
    identified_estimand=estimand,  
    method_name='backdoor.linear_regression')  
  
print(f'Estimate of causal effect (linear regression): {  
    estimate_lr.value}')
```

This gives us the following:

```
Estimate of causal effect (linear regression): 0.688147600764658
```

In this comparison, a complex DML model did a slightly better job than a simple linear regression, but it's hard to say whether this effect is anything more than noise (feel free to refit both models a couple of times to see how stable these results are; you can also re-generate or bootstrap the data to get even more reliable information).

Before we jump to refutation tests, I want to share one more thing with you. We had enough data to feed a complex DML estimator and the results are really good, but if you have a smaller dataset, a simpler method could be a better choice for you. If you're not sure what to choose, it's always a great idea to run a couple of quick experiments to see which methods are behaving better for your problem (or a similar problem represented by simulated data).

Step 4 – refute them!

As we've just seen, our DML model worked pretty well.

Let's check how robust it is.

We'll start by adding a random common cause:

```
random_cause = model.refute_estimate(  
    estimand=estimand,  
    estimate=estimate,  
    method_name='random_common_cause'  
)  
print(random_cause)
```

This results in the following:

```
Refute: Add a random common cause
Estimated effect:0.6981455013596706
New effect:0.6691543110272069
p value:0.13999999999999999
```

We see that there is a difference between the estimated effect (0.69814...) and the new effect (0.66915...), yet according to the provided p -value, this difference is not significant at the customary 0.05 level. If we accept this rule, we need to agree that the model has passed the test.

Let's try one more refutation test. This time, we'll replace our treatment variable with a random placebo variable:

```
placebo_refuter = model.refute_estimate(
    estimand=estimand,
    estimate=estimate,
    method_name='placebo_treatment_refuter'
)
print(placebo_refuter)
```

This gives us the following:

```
Refute: Use a Placebo Treatment
Estimated effect:0.6981455013596706
New effect:0.0
p value:2.0
```

This result is a clear indication that the placebo treatment had zero effect, which is the expected result for a healthy model.

It seems that our model passed both tests! Note that the first test (random common cause) belonged to the category of *invariant transformations*, while the second (placebo treatment) belonged to the category of *nullifying transformations*.

At the time of writing this chapter (September 2022), there are more refutation tests available for back-door criterion than there are for the front-door criterion.

For an up-to-date list of refutation tests available in DoWhy, visit <https://bit.ly/DoWhyRefutation>.

Now, let's switch to the experimental GCM API.

Extra – full example using the GCM API

The purpose of this subsection is to demonstrate the flexibility in structuring the causal inference process. APIs and tools will change with time, but a good understanding of the basics and the ability to translate your skills between different APIs, systems, or platforms are universal.

Note on the GCM API

At the time of writing (September 2022), the GCM API is still experimental. This means that there might be some changes to this API that will break the backward compatibility.

To work with the GCM API, we need to import `networkx` and the `gcm` subpackage:

```
import networkx as nx
from dowhy import gcm
```

We will reuse the data from our previous example. I'll put the data-generating code here as a refresher:

```
SAMPLE_SIZE = 1000

S = np.random.random(SAMPLE_SIZE)
Q = 0.2*S + 0.67*np.random.random(SAMPLE_SIZE)
X = 0.14*Q + 0.4*np.random.random(SAMPLE_SIZE)
Y = 0.7*X + 0.11*Q + 0.32*S
    + 0.24*np.random.random(SAMPLE_SIZE)
P = 0.43*X + 0.21*Y + 0.22*np.random.random(SAMPLE_SIZE)

df = pd.DataFrame(np.vstack([S, Q, X, Y, P]).T,
                  columns=['S', 'Q', 'X', 'Y', 'P'])
```

The next step is to generate the graph describing the structure of our data-generating process. The GCM API uses `nx.DiGraph` rather than GML strings as a graph representation.

Let's generate a graph:

```
edges = ['SQ', 'SY', 'QX', 'QY', 'XP', 'YP', 'XY']
graph_nx = nx.DiGraph([(edge[0],
                          edge[1]) for edge in edges])
```

We used list comprehension to automate the edge creation process. This is an analog of what we've done earlier with the two `for` loops. That said, defining a graph with NetworkX is simpler because we don't need to specify all the nodes explicitly.

To make sure that everything is as expected, let's plot the graph (Figure 7.6):

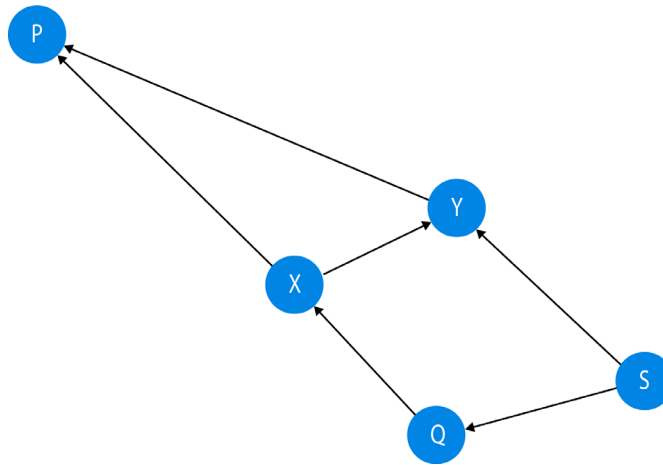


Figure 7.6 – A networkx representation of our graph

Now, let's define a causal model. It's very simple with the GCM API:

```
causal_model = gcm.InvertibleStructuralCausalModel(  
    graph_nx)
```

There are many different causal models available in the GCM API. We picked the invertible SCM, since this is the only model that allows us to generate counterfactuals without manually providing values for all noise variables.

Now it's time to define causal mechanisms for each variable:

```
causal_model.set_causal_mechanism('S', gcm.EmpiricalDistribution())  
causal_model.set_causal_mechanism('X', gcm.AdditiveNoiseModel(gcm.  
    ml.create_linear_regressor()))  
causal_model.set_causal_mechanism('Y', gcm.AdditiveNoiseModel(gcm.  
    ml.create_linear_regressor()))  
causal_model.set_causal_mechanism('P', gcm.AdditiveNoiseModel(gcm.  
    ml.create_linear_regressor()))  
causal_model.set_causal_mechanism('Q', gcm.AdditiveNoiseModel(gcm.  
    ml.create_linear_regressor()))
```

We use `gcm.EmpiricalDistribution()` for `S` because it's the only variable that does not have parents among endogenous variables. For all other variables, we set the causal mechanism to an additive causal model and use linear regression to model it.

We'll learn more about **additive noise models (ANMs)** in *Part 3, Causal Discovery*. For a detailed discussion on ANMs, check out Peters et al. (2017).

Let's fit the model to the data and estimate causal effects strengths:

```
gcm.fit(causal_model, df)
gcm.arrow_strength(causal_model, 'Y')
```

This results in the following:

```
{('Q', 'Y'): 0.0006632517083816319,
 ('S', 'Y'): 0.008486091866545382,
 ('X', 'Y'): 0.006866684034567223}
```

As you can see, the GCM API returns estimates for all the variables with the incoming edges to the variable of interest (Y in our case; see *Figure 7.6* for reference).

These results are different from what we've seen before. The reason for this is that the GCM API by default returns results in terms of the outcome variable variance change given we remove the edge from the source variable. This behavior can be changed and you can define your own estimation functions. To learn more, refer to the GCM API documentation, available here: <https://bit.ly/DoWhyArrowStrength>.

A great feature of the GCM API is that when you create and fit a model, you can easily answer different types of causal queries using this model.

For instance, you can generate counterfactuals:

```
gcm.counterfactual_samples(
    causal_model,
    {'X': lambda x: .21},
    observed_data=pd.DataFrame(data=dict(X=[.5],
    Y=[.75], S=[.5], Q=[.4], P=[.34])))
```

This gives us the following result (*Figure 7.7*):

	S	Q	X	Y	P
0	0.5	0.4	0.21	0.539558	0.169005

Figure 7.7 – Counterfactuals from our GCM when X is set to 0.21

To learn more about the GCM API, please refer to the documentation.

Wrapping it up

In this chapter, we discussed the Python causal ecosystem. We introduced the DoWhy and EconML libraries and practiced the four-step causal inference process using DoWhy's CausalModel API. We learned how to automatically obtain estimands and how to use different types of estimators to compute causal effect estimates. We discussed what refutation tests are and how to use them in practice. Finally, we introduced DoWhy's experimental GCM API and showed its great capabilities when it comes to answering various causal queries. After working through this chapter, you have the basic skills to apply causal inference to your own problems. Congratulations!

In the next chapter, we'll summarize common assumptions for causal inference and discuss some limitations of the causal inference framework.

References

Bates, S., Hastie, T., & Tibshirani, R. (2021). *Cross-validation: what does it estimate and how well does it do it?*. arXiv preprint. <https://doi.org/10.48550/ARXIV.2104.00673>

Battocchi, K., Dillon, E., Hei, M., Lewis, G., Oka, P., Oprescu, M., & Syrgkanis, V. (2019). *EconML: A Python Package for ML-Based Heterogeneous Treatment Effects Estimation*. <https://github.com/microsoft/EconML>

Blobaum, P., Götz, P., Budhathoki, K., Mastakouri, A., & Janzing, D. (2022). *DoWhy-GCM: An extension of DoWhy for causal inference in graphical causal models*. arXiv.

Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., & Robins, J. (2016). *Double/Debiased Machine Learning for Treatment and Causal Parameters*. arXiv preprint. <https://doi.org/10.48550/ARXIV.1608.00060>

Molak, A. (2022, September 27). *Causal Python: 3 Simple Techniques to Jump-Start Your Causal Inference Journey Today*. Towards Data Science. <https://towardsdatascience.com/causal-kung-fu-in-python-3-basic-techniques-to-jump-start-your-causal-inference-journey-tonight-ae09181704f7>

Peters, J., Janzing, D., & Schölkopf, B. (2017). *Elements of Causal Inference: Foundations and Learning Algorithms*. MIT Press.

Popper, K. (1935). *Logik der Forschung*. Springer.

Popper, K. (1959). *The Logic of Scientific Discovery*. Basic Books.

Sharma, A. & Kiciman, E. (2020). *DoWhy: An End-to-End Library for Causal Inference*. arXiv preprint. arXiv:2011.04216.

Shimoni, Y., Karavani, E., Ravid, S., Bak, P., Ng, T. H., Alford, S. H., Meade, D., & Goldschmidt, Y. (2019). *An Evaluation Toolkit to Guide Model Selection and Cohort Definition in Causal Inference*. arXiv preprint. arXiv:1906.00442.